

处理器体系结构

现代微处理器可以称得上是人类创造出的最复杂的系统之一。一块手指甲大小的硅片上，可以容纳一个完整的高性能处理器、大的高速缓存，以及用来连接到外部设备的逻辑电路。从性能上来说，今天在一块芯片上实现的处理器已经使 20 年前价值 1000 万美元、房间那么大的超级计算机相形见绌了。即使是在像手机、导航系统和可编程恒温器这样的日常设备中的嵌入式处理器，也比早期计算机开发者所能想到的强大得多。

到目前为止，我们看到的计算机系统只限于机器语言程序级。我们知道处理器必须执行一系列指令，每条指令执行某个简单操作，例如两个数相加。指令被编码为由一个或多个字节序列组成的二进制格式。一个处理器支持的指令和指令的字节级编码称为它的指令集体系结构(Instruction-Set Architecture, ISA)。不同的处理器“家族”，例如 Intel IA32 和 x86-64、IBM/Freescale Power 和 ARM 处理器家族，都有不同的 ISA。一个程序编译成在一种机器上运行，就不能在另一种机器上运行。另外，同一个家族里也有很多不同型号的处理器。虽然每个厂商制造的处理器性能和复杂性不断提高，但是不同的型号在 ISA 级别上都保持着兼容。一些常见的处理器家族(例如 x86-64)中的处理器分别由多个厂商提供。因此，ISA 在编译器编写者和处理器设计人员之间提供了一个概念抽象层，编译器编写者只需要知道允许哪些指令，以及它们是如何编码的；而处理器设计者必须建造出执行这些指令的处理器。

本章将简要介绍处理器硬件的设计。我们将研究一个硬件系统执行某种 ISA 指令的方式。这会使你能更好地理解计算机是如何工作的，以及计算机制造商们面临的技术挑战。一个很重要的概念是，现代处理器的实际工作方式可能跟 ISA 隐含的计算模型大相径庭。ISA 模型看上去应该是顺序指令执行，也就是先取出一条指令，等到它执行完毕，再开始下一条。然而，与一个时刻只执行一条指令相比，通过同时处理多条指令的不同部分，处理器可以获得更高的性能。为了保证处理器能得到同顺序执行相同的结果，人们采用了一些特殊的机制。在计算机科学中，用巧妙的方法在提高性能的同时又保持一个更简单、更抽象模型的功能，这种思想是众所周知的。在 Web 浏览器或平衡二叉树和哈希表这样的信息检索数据结构中使用缓存，就是这样的例子。

你很可能永远都不会自己设计处理器。这是专家们的任务，他们工作在全球不到 100 家的公司里。那么为什么你还应该了解处理器设计呢？

- 从智力方面来说，处理器设计是非常有趣而且很重要的。学习事物是怎样工作的有其内在价值。了解作为计算机科学家和工程师日常生活一部分的一个系统的内部工作原理(特别是对很多人来说这还是个谜)，是件格外有趣的事情。处理器设计包括许多好的工程实践原理。它需要完成复杂的任务，而结构又要尽可能简单和规则。
- 理解处理器如何工作能帮助理解整个计算机系统如何工作。在第 6 章，我们将讲述存储器系统，以及用来创建很大的内存映像同时又有快速访问时间的技术。看看处理器端的处理器——内存接口，会使那些讲述更加完整。

- 虽然很少有人设计处理器，但是许多人设计包含处理器的硬件系统。将处理器嵌入到现实世界的系统中，如汽车和家用电器，已经变得非常普通了。嵌入式系统的设计者必须了解处理器是如何工作的，因为这些系统通常在比桌面和基于服务器的系统更低抽象级别上进行设计和编程。
- 你的工作可能就是处理器设计。虽然生产处理器的公司很少，但是研究处理器的设计人员队伍已经非常巨大了，而且还在壮大。一个主要的处理器设计的各个方面大约涉及 1000 多人。

本章首先定义一个简单的指令集，作为我们处理器实现的运行示例。因为受 x86-64 指令集的启发，它被俗称为“x86”，所以我们称我们的指令集为“Y86-64”指令集。与 x86-64 相比，Y86-64 指令集的数据类型、指令和寻址方式都要少一些。它的字节级编码也比较简单，机器代码没有相应的 x86-64 代码紧凑，不过设计它的 CPU 译码逻辑也要简单一些。虽然 Y86-64 指令集很简单，它仍然足够完整，能让我们写一些处理整数的程序。设计一个实现 Y86-64 的处理器要求我们解决许多处理器设计者同样会面对的问题。

接下来会提供一些数字硬件设计的背景。我们会描述处理器中使用的基本构件块，以及它们如何连接起来和操作。这些介绍是建立在第 2 章对布尔代数和位级操作的讨论的基础上的。我们还将介绍一种描述硬件系统控制部分的简单语言，HCL (Hardware Control Language, 硬件控制语言)。然后，用它来描述我们的处理器设计。即使你已经有了一些逻辑设计的背景知识，也应该读读这个部分以了解我们的特殊符号表示方法。

作为设计处理器的第一步，我们给出一个基于顺序操作、功能正确但是有点不实用的 Y86-64 处理器。这个处理器每个时钟周期执行一条完整的 Y86-64 指令。所以它的时钟必须足够慢，以允许在一个周期内完成所有的动作。这样一个处理器是可以实现的，但是它的性能远远低于同样的硬件应该能达到的性能。

以这个顺序设计为基础，我们进行一系列的改造，创建一个流水线化的处理器 (pipelined processor)。这个处理器将每条指令的执行分解成五步，每个步骤由一个独立的硬件部分或阶段 (stage) 来处理。指令步经流水线的各个阶段，且每个时钟周期有一条新指令进入流水线。所以，处理器可以同时执行五条指令的不同阶段。为了使这个处理器保留 Y86-64 ISA 的顺序行为，就要求处理很多冒险或冲突 (hazard) 情况，冒险就是一条指令的位置或操作数依赖于其他仍在流水线中的指令。

我们设计了一些工具来研究和测试处理器设计。其中包括 Y86-64 的汇编器、在你的机器上运行 Y86-64 程序的模拟器，还有针对两个顺序处理器设计和一个流水线化处理器设计的模拟器。这些设计的控制逻辑用 HCL 符号表示的文件描述。通过编辑这些文件和重新编译模拟器，你可以改变和扩展模拟器行为。我们还提供许多练习，包括实现新的指令和修改机器处理指令的方式。还提供测试代码以帮助你评价修改的正确性。这些练习将极大地帮助你理解所有这些内容，也能使你更理解处理器设计者面临的许多不同的设计选择。

网络旁注 ARCH:VLOG 给出了用 Verilog 硬件描述语言描述的流水线化的 Y86-64 处理器。其中包括为基本的硬件构建块和整个的处理器结构创建模块。我们自动地将控制逻辑的 HCL 描述翻译成 Verilog。首先用我们的模拟器调试 HCL 描述，能消除很多在硬件设计中会出现的棘手的问题。给定一个 Verilog 描述，有商业和开源工具来支持模拟和逻辑合成 (logic synthesis)，产生实际的微处理器电路设计。因此，虽然我们在此花费大部分精力创建系统的图形和文字描述，写软件的时候也会花费同样的精力，但是这些设计能够自动地合成，这表明我们确实在创建一个能够用硬件实现的系统。